Healthcare Appointment & Follow-up Manager

Technical Submission Documentation

Database Schema Design • API Design • Code Structure • AI / Engineering Highlights

Purpose: Present the project so a technical reviewer or automated code-review system can quickly understand the problem, architecture, data model, API surface, key engineering decisions, and implementation boundaries.

1. Executive Technical Summary

The Healthcare Appointment & Follow-up Manager is a Django-based healthcare platform organized around a reliable appointment lifecycle: patient registration → doctor discovery → slot availability → temporary slot hold → symptom submission → appointment confirmation → doctor consultation → post-visit summary.

The project is more than a basic CRUD booking application. Its engineering focus is safe appointment scheduling under concurrent requests, role-aware access, separation of domain logic into Django apps, and AI-assisted pre- and post-visit information. The supplied brief explicitly evaluates database schema design, API design and code structure, together with LLM, email, and Google Calendar integration.

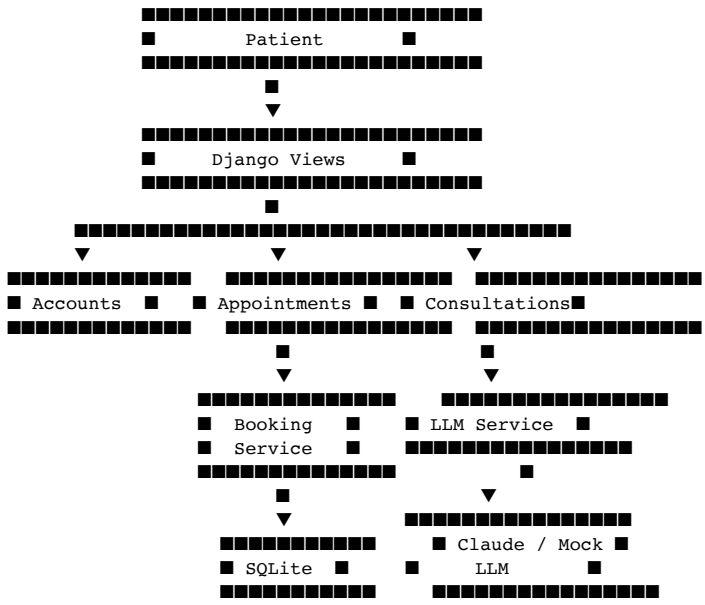
Reviewer takeaway: The strongest implemented engineering area is the multi-layer appointment conflict strategy: transaction handling + row locking + a database constraint + graceful IntegrityError handling.

2. Requirement → Implementation Map

This table maps the requested capabilities to the implementation described in the project documentation, making the relationship between requirements and code easy to identify.

Requirement	Implementation / Evidence
Role-based authentication	Custom User with patient / doctor / admin roles; accounts app handles authentication and access control.
Doctor management	DoctorProfile stores specialization, schedule, slot duration, bio and active state.
Doctor leave	Leave entity stores doctor/date/reason with a unique doctor/date rule.
Appointment booking	Appointment entity plus booking flow and slot-hold mechanism.
Double-booking prevention	transaction.atomic(), select_for_update(), active-slot conditional uniqueness, IntegrityError handling.
Dynamic availability	GET /appointments/api/slots/ returns available slots as JSON.
Pre-visit AI	SymptomForm + generate_pre_visit_summary().
Post-visit AI	PostVisitNote + generate_post_visit_summary().
LLM resilience	LLM status tracking and Mock LLM/failure-handling design.
Modular code structure	Separate accounts, doctors, appointments, consultations and notifications_app Django apps.

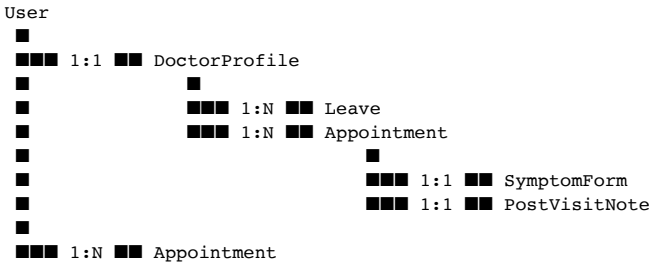
3. System Architecture



The application uses a modular Django architecture. HTML templates provide the primary UI, while JavaScript fetch() supports asynchronous slot availability. Authentication, scheduling, consultation/AI logic, and integration responsibilities are separated.

4. Database Schema Design

The database uses SQLite through Django ORM and is centered on User → DoctorProfile → Appointment → Consultation, with Leave supporting scheduling and notification/integration entities around the appointment lifecycle.

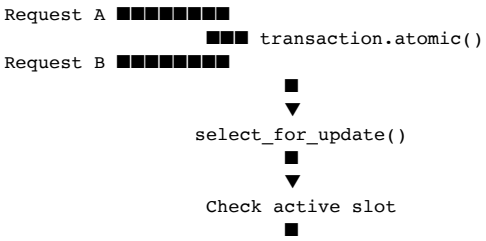


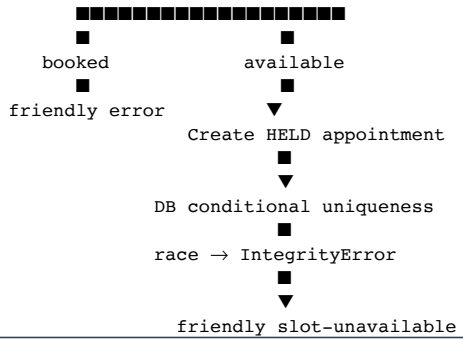
4.1 Core Entities

Entity	Key fields / purpose
User	id, username, email, role, phone_number, date_of_birth. Roles: patient / doctor / admin.
DoctorProfile	OneToOne User; specialization; working hours; slot duration; working days; bio; is_active.
Leave	doctor, date, reason, created_at. (doctor, date) is unique.
Appointment	patient, doctor, date, start/end time, status, hold_expires_at, calendar event IDs, timestamps.
SymptomForm	appointment OneToOne; symptoms; urgency; chief complaint; suggested questions; LLM status; raw response.
PostVisitNote	appointment OneToOne; clinical notes; prescription; medications JSON; patient summary; follow-up; LLM status.

5. Key Engineering Challenge: Double-Booking Prevention

Availability checks alone are insufficient for concurrent requests. Two users can observe the same slot as free before either request commits. The documented design therefore uses multiple protection layers.





Why this stands out: The design combines application-level synchronization with a database-level consistency guarantee. The conditional uniqueness rule applies to active held/confirmed appointments.

6. Appointment Lifecycle & Slot Hold

```
GET /appointments/api/slots/
          ■
          ▼
Patient selects slot
          ■
          ▼
POST /appointments/book/
          ■
          ▼
hold_slot()
          ■
          ■■■ status = HELD
          ■■■ hold_expires_at = set
          ■
          ▼
Patient submits symptom form
          ■
          ▼
LLM generates pre-visit information
          ■
          ▼
Appointment → CONFIRMED
          ■
          ▼
Doctor consultation
```

7. API Design

Primary JSON endpoint: GET /appointments/api/slots/

```
GET /appointments/api/slots/?doctor_id=2&date=2026-08-25

{
  "doctor_id": 2,
  "date": "2026-08-25",
  "slots": ["09:00", "09:30", "10:00", "10:30"]
}
```

The endpoint validates doctor/date inputs, calculates working slots, removes occupied or unavailable slots, and returns the remaining slots as JSON.

Condition	HTTP	Response
Missing doctor_id/date	400	doctor_id and date query parameters are required
Invalid doctor	404	Doctor not found
Invalid date	400	Invalid date format. Use YYYY-MM-DD

7.1 Main Application Routes

Method	Endpoint	Purpose
GET/POST	/accounts/register/	Patient registration
GET	/accounts/login/	Login
POST	/accounts/logout/	Logout
GET	/accounts/dashboard/	Role-based dashboard
GET	/appointments/api/slots/	Available appointment slots
GET	/appointments/doctors/	Doctor search
GET	/appointments/doctors/<id>/	Doctor details and slots
POST	/appointments/book/	Hold/book an appointment
GET	/appointments/mine/	Patient appointments
GET	/appointments/doctor/	Doctor appointments
POST	/appointments/<id>/cancel/	Cancel appointment

8. Code Structure & Separation of Concerns

```
Clinic_AI_Healthcare_Appointment/
■■■ accounts/
■   ■■■ models.py
■   ■■■ views.py
■   ■■■ forms.py
■   ■■■ decorators.py
■   ■■■ urls.py
■   ■■■ admin.py
■■■ doctors/
■   ■■■ models.py
■   ■■■ views.py
■   ■■■ admin.py
■■■ appointments/
■   ■■■ models.py
■   ■■■ views.py
■   ■■■ services.py
■   ■■■ urls.py
■■■ consultations/
■   ■■■ models.py
■   ■■■ views.py
■   ■■■ llm_service.py
■■■ notifications_app/
■■■ clinic_project/
■■■ templates/
■■■ static/
■■■ manage.py
■■■ requirements.txt
■■■ db.sqlite3
```

App	Responsibility
accounts	Custom User, authentication, roles, registration, dashboard routing, access control
doctors	Doctor profiles, specialization, schedule, slot duration, leave
appointments	Appointment model, availability, booking, slot holding, cancellation, concurrency protection
consultations	Symptoms, AI pre-visit summary, clinical notes, prescriptions, post-visit summary
notifications_app	Notification/integration layer for email, calendar and background processing

8.1 Why the Service Layer Matters

The booking logic is isolated in the appointment service layer rather than being treated as ordinary view logic. This makes concurrency-sensitive behavior easier to reason about, reuse and test. The LLM service is similarly isolated in consultations/llm_service.py, keeping provider-specific AI behavior out of the main consultation flow.

9. AI Architecture

```
Patient symptoms
      ■
      ▼
generate_pre_visit_summary()
      ■
      ▼
Anthropic Claude API / Mock LLM
      ■
      ▼
Structured JSON
      ■
      ▼
SymptomForm

Clinical notes + prescription
      ■
      ▼
generate_post_visit_summary()
      ■
      ▼
Anthropic Claude API / Mock LLM
      ■
      ▼
Patient summary + medication + follow-up
```

The pre-visit path produces urgency, chief complaint and suggested questions. The post-visit path produces patient-friendly summary information, medication information and follow-up steps. LLM status fields distinguish pending, successful and failed processing.

AI design principle: AI is an assistive layer around the consultation workflow. The documented design treats LLM failure as a recoverable application condition rather than a reason to crash or block the core appointment flow.

10. Design Decisions

Decision	Reason
SQLite + Django ORM	Appropriate for the development/demo environment and specified database technology.
Separate Django apps	Keeps authentication, doctors, appointments, consultations and integrations independently maintainable.
JSONField for AI data	Fits semi-structured outputs such as suggested questions and medication lists.
Booking service layer	Keeps concurrency-sensitive logic out of views and makes it reusable/testable.
Database constraint	Provides a final consistency guarantee beyond application-level availability checks.
Mock LLM mode	Keeps the AI workflow demoable without requiring external credentials.

11. Reviewer-Oriented Engineering Highlights

• **Concurrency:** booking is designed for simultaneous requests, not only sequential demo usage.

• **Data integrity:** database constraints complement application-level validation.

• **State modeling:** appointment statuses make the lifecycle explicit: held → confirmed → completed, with cancellation states.

• **Separation of concerns:** domain responsibilities are split across focused Django apps and service modules.

• **AI integration:** the LLM is isolated behind explicit service functions and status tracking.

• **API clarity:** the slot availability endpoint has defined parameters, JSON output and explicit error responses.

• **Explainability:** major design decisions are documented rather than only listing technologies.

12. Implementation Status & Accuracy Boundary

This section is deliberately explicit so the submission does not overclaim functionality. The reviewed repository documentation identifies the following areas as implemented or present, while some integration requirements were not evident in the reviewed repository.

Area	Status in reviewed repository
Custom User / roles	Implemented
DoctorProfile	Implemented
Leave model	Implemented
Appointment model	Implemented
Double-booking protection	Implemented
Slot holding	Implemented
SymptomForm	Implemented
PostVisitNote	Implemented
LLM / Mock LLM service	Implemented
Dynamic slot JSON API	Implemented
notifications_app	Present, but incomplete in reviewed repository
EmailLog / MedicationReminder	Not currently evident in reviewed repository
Google Calendar persistence/service	Not currently evident in reviewed repository

Submission rule: Only claim an external integration as complete when the corresponding code, configuration and documentation are present in the final GitHub repository.

13. Verification-Friendly GitHub Evidence

To make the technical claims easy to verify, keep the relevant implementation files discoverable: appointments/services.py for booking logic, appointments/models.py for appointment constraints, appointments/views.py and urls.py for API behavior, doctors/models.py for slot generation, consultations/llm_service.py for AI integration, and the model files for the database entities.

A concise README, architecture diagram, database diagram, screenshots/demo video, and meaningful tests would further reduce the effort required for a reviewer or automated system to validate the project.

14. Final Technical Positioning

The project should be presented as a healthcare workflow and scheduling system with AI-assisted consultation support—not simply as an appointment CRUD application. The strongest technical narrative is the combination of a relational data model, explicit appointment states, concurrency-safe slot booking, modular Django architecture, a focused availability API, and isolated LLM services.

One-line reviewer summary: “A Django healthcare appointment platform that combines role-based workflows and AI-assisted consultations with concurrency-safe appointment scheduling and database-level protection against double-booking.”